

seL4 elfloader

Page Content

- [General](#)
- [elfloader in RISC-V](#)
 - [Entry](#)
 - [C code entry](#)
- [Copying the elf files from archive to \(physical\) memory](#)
 - [Setting up the page tables for executing seL4 \(in virtual mode\)](#)
- [Conclusion for our ROM code](#)

Sub-Pages

- [Page tables created](#)

General

- elfloader is part of the [seL4 boot process](#)
- The elf-loader can be found in https://github.com/seL4/seL4_tools/blob/master/elfloader-tool
- From <https://sel4.discourse.group/t/boot-elf-loading-and-initial-task/199>:

ARM and RISC-V both boot via the intermediary elfloader which is responsible for booting multiple cores, unpacking the ELF for the kernel and the initial task into physical memory a small amount of information about the initial task and device tree (although the form this currently takes is fairly limited). For x86, the kernel boots directly from multiboot and the boot code within the kernel is responsible for ELF loading the initial task and initialising additional cores.

The interface between the kernel and whatever starts it is fairly limited and can make it difficult to start the kernel via something other than elf loader (such as a ROM-based firmware loader) without needing to maintain a fork of the kernel itself. The documentation for this process is also limited.

For static systems, where there is a static set of task ELF files that are loaded at system startup, this can also lead to both elf loader and the root task being responsible for loading ELF binaries into memory.

I'm personally a fan of the way that elf loader separates the concern of initializing physical memory from the kernel's responsibility of creating the initial kernel objects and capabilities for the root task and think we could better generalize this. I think we should consider making the kernel's boot responsibilities more consistent across architectures and better defined and documented.

Fundamentally, the only responsibility the kernel *needs* to have is to initialize its global and per-core internal state and to create the initial capabilities and kernel objects and set up the capability space and virtual address space for the initial task. Much of what occupies the initial state of the initial task could be passed through directly from the whatever initializes the kernel, all the kernel needs to be able to add is the information describing how the initial capability space is structured.

The build process of seL4_test can be summarized as:

1. build the seL4 kernel elf
2. build the seL4_test user elf
3. store these two elfs in a cpio archive
4. build the elf-loader elf which contains this archive as a blob
5. build the proxy kernel elf which contains the elf-loader elf as a blob

Because of the requirements of the proxy kernel in RISC-V build process the elf-loader elf needs to meet these two requirements:

1. the code entry point has to be at the very beginning of the first section (sorted by section addresses)
2. it must not contain any re-locations (a linker/loader would have to resolve before executing the code)

elfloader in RISC-V

Entry

- code runs in S-Mode with MMU off
- the proxy kernel passes parameters:
 - a0 = hart id
 - a1 = pointer to the device tree
- the execution of the elf loader begins in elfloader-tool/src/arch-riscv/crt0.S.

```

.section ".text.start"

_start:

.option push
.option norelax
1:auipc gp, %pcrel_hi(__global_pointer$)
    addi gp, gp, %pcrel_lo(1b)
.option pop

/* mhartid value is in a0 */
la sp, (elfloader_stack_alloc + BIT(12))

la s0, main
jr s0

```

Notes:

- The .options directives target the tool-chain (<https://gcc.gnu.org/onlinedocs/gcc/RISC-V-Options.html>):
 1. .option push = save the current tool-chain settings
 2. .option norelax: de-activate the relaxation
 3. .option pop: activate the previously pushed settings again
- This temporary deactivation of the relaxation is required in order to prevent the linker from interfering with the initialization of the gp/x3 register ("global pointer").
 - Linker relaxation assumes a properly initialized gp in the first place.
 - A very good explanation of this contained in: <https://www.sifive.com/blog/all-aboard-part-3-linker-relaxation-in-riscv-toolchain>
- The linker script makes sure that _start is to be found at the very beginning:

```

ENTRY( _start )

SECTIONS
{
    . = 0x0000000080400000;

    /* text: Program code section */
    .start :
    {
        *(.text.start)
    }
}

```

- Important: as indicated by the linker file above, the elf-loader is linked to run at 0x80400000. During the build process of the proxy kernel the elf-loader elf is "imported" as a blob at an arbitrary address X. After execution the proxy kernel simply will jump to X (hence the requirement that its entry point is at the very beginning). Because of this arbitrary location X, the elf-loader has to be built such a way that the resulting binary is position independent (= e.g. it must not rely to be actually executed at location 0x80400000).

C code entry

- code is still running on physical addresses in S-Mode, the MMU is not enabled at this point.
- The execution continues in elfloader-tool/src/arch-riscv/boot.c

```

void main(int hardid, unsigned long dtb)
{
    (void) hardid;
    printf("ELF-loader started on\n");

    printf("  paddr=[%p..%p]\n", _start, _end - 1);
    /* Unpack ELF images into memory. */
    load_images(&kernel_info, &user_info, 1, &num_apps);
    if (num_apps != 1) {
        printf("No user images loaded!\n");
        abort();
    }

    map_kernel_window(&kernel_info);

    printf("Jumping to kernel-image entry point...\n\n");

    asm volatile("sfence.vma");

    asm volatile(
        "csrw sptbr, %0\n"
        :
        : "r" (vm_mode | (uintptr_t)l1pt >> RISCV_PGSHIFT)
        :
    );

    ((init_riscv_kernel_t)kernel_info.virt_entry)(user_info.phys_region_start,
                                                user_info.phys_region_end, user_info.phys_virt_offset,
                                                user_info.virt_entry, 0, dtb);

    /* We should never get here. */
    printf("Kernel returned back to the elf-loader.\n");
}

```

Notes:

- When this main is executed the printf() is ultimately backed by the proxy kernel's SBI interface
- The two elf images (kernel and user (e.g. seL4_test driver)) are extracted from the included archive and copied to physical memory (described below)
- MMU is set up
 - a page table that covers the kernel is created (described below)
 - MMU is activated for the currently running S-Mode
 - RISC-V spec V1.9 and 1.10 are slightly different here
 - V1.9: Supervisor Page-Table Base Register (sptbr) contains page table base, writing *mode* to VM[0:4] in mstatus enables MMU
 - V1.10: Supervisor Address Translation and Protection Register (satp) contains both *mode* and *table base address*
 - looks like the code above uses V1.10, but still uses old register name name sptbr instead of satp
- Control is passed to seL4 with the following arguments:
 - a0 = phys_region_start: the physical address of the user image (= this is the physical address the kernel was copied to plus the kernel's image size plus alignment (see below))
 - a1 = phys_region_end: the previous argument plus the image size of the user image (page aligned)
 - a2 = phys_virt_offset: the difference between physical and virtual position of the user image
 - a3 = virt_entry: the entry point of the user image
 - a4 = the hart id
 - a5 = the address of the device tree

Copying the elf files from archive to (physical) memory

This is implemented with the function `load_images` in `elfloader-tool/src/common.c`. It uses the two helper functions:

- `load_elf` in `elfloader-tool/src/common.c`
- `elfMemoryBounds` in `elfloader-tool/src/binaries/elf/elf.c`

```

/**
 * Determine the memory bounds of an ELF file
 *
 * @param elfFile Pointer to a valid ELF header
 * @param phys If true return bounds of physical memory, otherwise return
 *           bounds of virtual memory
 * @param min Pointer to return value of the minimum
 * @param max Pointer to return value of the maximum
 *
 * \return true on success. false on failure, if for example, it is an invalid ELF file
 */
int elf_getMemoryBounds(void *elfFile, int phys, uint64_t *min, uint64_t *max);

/* FOR WIKI:
 * elfMemoryBounds scans all program headers of the given elf file and returns a pair
 * of addresses which mark the minimum and the maximum address occupied by the union
 * of all the program headers. The second argument (0 or 1) determines if this process
 * is done for the physical (1) or virtual (0) addresses of the program headers. */

```

```

/*
 * Load an ELF file into physical memory at the given physical address.
 *
 * Return the byte past the last byte of the physical address used.
 */
static paddr_t load_elf(const char *name, void *elf, paddr_t dest_paddr,
                      struct image_info *info, int keep_headers,
                      __attribute__((unused)) unsigned long size,
                      __attribute__((unused)) const char *hash)

/* FOR WIKI:
 * The name, size and hash argument are used either for debug output or optional hash
 * functionality.
 * This function copies all program headers to the given physical address. More specifically:
 * a) the program header with the smallest virtual address is copied to dest_paddr
 * b) all other program headers are copied to physical memory such that their relative virtual
 *    placement to the program header with the smallest virtual address is not changed.
 * If the argument keep_headers is true then additionally the table of the program headers from
 * the elf file is copied to a separate page (page aligned) right after the program header with
 * the maximum virtual address.
 * The function returns the smallest physical address (page aligned) which has not been touched
 * during the elf copy operation.
 * In addition the output parameter info is initialized with the following code: */

    elf_getMemoryBounds(elf, 0, &min_vaddr, &max_vaddr);
    max_vaddr = ROUND_UP(max_vaddr, PAGE_BITS);
    image_size = (size_t)(max_vaddr - min_vaddr);
    info->phys_region_start = dest_paddr;
    info->phys_region_end = dest_paddr + image_size;
    info->virt_region_start = (vaddr_t)min_vaddr;
    info->virt_region_end = (vaddr_t)max_vaddr;
    info->virt_entry = (vaddr_t)elf_getEntryPoint(elf);
    info->phys_virt_offset = dest_paddr - (vaddr_t)min_vaddr;

/* So the output parameter info describes:
 * - the physical and virtual region of the elf as well as their relative offset
 * - the entry point of the elf
 * Important: these regions do not consider the optional placement of the program headers in
 * the page right after the elf image. */

```

```

void load_images(struct image_info *kernel_info, struct image_info *user_info,
                int max_user_images, int *num_images)
{
    int i;
    uint64_t kernel_phys_start, kernel_phys_end;
    paddr_t next_phys_addr;
    const char *elf_filename;
    unsigned long unused;

    /* Load kernel. */
    void *kernel_elf = cpio_get_file(_archive_start, "kernel.elf", &unused);
    if (kernel_elf == NULL) {
        printf("No kernel image present in archive!\n");
        abort();
    }
    if (elf_checkFile(kernel_elf)) {
        printf("Kernel image not a valid ELF file!\n");
        abort();
    }

    elf_getMemoryBounds(kernel_elf, 1, &kernel_phys_start, &kernel_phys_end);
    next_phys_addr = load_elf("kernel", kernel_elf,
                            (paddr_t)kernel_phys_start, kernel_info, 0, unused, "kernel.bin");

    /*
     * Load userspace images.
     *
     * We assume (and check) that the kernel is the first file in the archive,
     * and then load the (n+1)'th file in the archive onto the (n)'th CPU.
     */
    (void)cpio_get_entry(_archive_start, 0, &elf_filename, &unused);
    if (strcmp(elf_filename, "kernel.elf") != 0) {
        printf("Kernel image not first image in archive.\n");
        abort();
    }
    *num_images = 0;
    for (i = 0; i < max_user_images; i++) {
        /* Fetch info about the next ELF file in the archive. */
        void *user_elf = cpio_get_entry(_archive_start, i + 1,
                                        &elf_filename, &unused);

        if (user_elf == NULL) {
            break;
        }

        /* Load the file into memory. */
        next_phys_addr = load_elf(elf_filename, user_elf,
                                next_phys_addr, &user_info[*num_images], 1, unused, "app.bin");
        *num_images = i + 1;
    }
}

```

Comments:

- the smallest physical address of all the program headers of the kernel's elf is determined
- the kernel's elf is copied to this address
- the program header table of the kernel's program headers is **NOT** placed after the kernel image
- the user image (there is only one in for seL4_test) is copied to physical memory right after the kernel
- the program header table of the user image's program headers **IS** placed (page aligned) right after the user image
 - TODO: clarify how (or if) the kernel is using this data (or is making it available to the initial thread).

Setting up the page tables for executing seL4 (in virtual mode)

This is implemented in the function `map_kernel_window` in `elfloader-tool/arch-riscv/boot.c`

```

void
map_kernel_window(struct image_info *kernel_info)
{
    uint64_t i;
    uint32_t l1_index;

    // first create a complete set of 1-to-1 mappings for all of memory. this is a brute
    // force way to ensure this elfloader is mapped into the new address space
    for(i = 0; i < PTES_PER_PT; i++)
    {
        l1pt[i] = PTE_CREATE((uint64_t)(i << PPN_HIGH));
    }
    //Now create any neccessary entries for the kernel vaddr->paddr
    l1_index = GET_PT_INDEX(kernel_info->virt_region_start, PT_LEVEL_1);

    /* Check if aligned to top level page table. For Sv32, 2MiB. For Sv39, 1GiB */
    if (VIRT_PHYS_ALIGNED(kernel_info->virt_region_start, kernel_info->phys_region_start, PTE_LEVEL_BITS))
    {
        for (int page = 0; l1_index < PTES_PER_PT; l1_index++, page++)
        {
            l1pt[l1_index] = PTE_CREATE_LEAF((seL4_Word)(kernel_info->phys_region_start + (page <<
PTE_LEVEL_BITS)));
        }
    }
    #if CONFIG_PT_LEVELS == 3
    else if (VIRT_PHYS_ALIGNED(kernel_info->virt_region_start, kernel_info->phys_region_start, PT_LEVEL_2_BITS))
    {
        uint32_t l2_index = GET_PT_INDEX(kernel_info->virt_region_start, PT_LEVEL_2);
        l1pt[l1_index] = PTE_CREATE_NEXT((seL4_Word)l2pt);

        for (int page = 0; l2_index < PTES_PER_PT; l2_index++, page++)
        {
            l2pt[l2_index] = PTE_CREATE_LEAF(kernel_info->phys_region_start + (page << PT_LEVEL_2_BITS));
        }
    }
    #endif
    else
    {
        printf("Kernel not properly aligned\n");
        abort();
    }
}

```

Comments:

- This function supports Sv32 and Sv39
- As (sole) input it gets a parameter of type `struct image_info` which was created by `load_elf`
- The start addresses in physical and virtual memory need to be aligned
 - Sv32: 2 MB
 - Sv39: 1 GB
 - Sv39: 2 MB (with `CONFIG_PT_LEVELS` set to 3 which means a two level page table)
- If allowed by the alignment a one-level page table is constructed. (Sv32: 2MB / Sv39: 1GB)
- If not then a second level is added to the page table (turning one entry in the first level from a "leaf" to a "next" entry - provided `CONFIG_PT_LEVELS` set to 3)
- [Page tables created](#) contains an example of the page tables created by the elf-loader's "map_kernel_window"

Conclusion for our ROM code

- ELF Loader can be dropped completely, seL4 code segment can be placed in ROM directly, see als [TRENTOS-M Secure Boot and Update#ROMLayout](#)
- MMU tables for kernel can be generated at compile time
- boot loader runs in M-Mode in physical addresses and will:
 - initialize seL4 data segment in RAM (ie copy from ROM to RAM, or see that seL4 does not need any data segment at all)
 - invalidate all caches
 - set page tables and enable MMU
 - start seL4
- ToDo:

- check if M-Mode can setup and enable MMU for S-Mode, so we could have an atomic exit from the boot loader that enters seL4 in S-Mode with MMU on.
- user image can also be ROM code and contain the next boot loader stage.
- remove all DTB and all parsing from seL4 RISC-V code and use compile-time generated header files (from the DTB) for this instead.